TFG · SONDA PASIVA DE DETECCIÓN PQC · TAREA 5

libpcap en la sonda

Qué funciones usa, por qué, y lo que conviene añadir para la captura en vivo

Pablo Marugán Bastante · GITST, EPS-UAM · Tutor: Jorge E. López de Vergara | **04/11/2026** | Tutoría del miércoles 04/11/2026

1. Situación de la tarea

> **ESTADO**
>
> No se ha localizado el enunciado de la práctica de ARP de Arquitectura de Redes II. En su lugar se documenta **cómo se aplica libpcap en la sonda**, que era el objetivo de repasar la práctica, junto con lo que de ella ya se revisó en la tutoría 1.

2. El ciclo de vida de una captura

Función	Qué hace	Cómo se usa en `sonda.c`
`pcap_open_offline()`	Abre un fichero pcap/pcapng	Opción `-f`. Es como se han hecho todas las pruebas: misma traza, mismo resultado, comparable con Wireshark
`pcap_open_live()`	Abre una interfaz para capturar en vivo	Opción `-i`. **snaplen 65535** para no cortar el `key_share` (la clave PQ son 1216 B); modo **promiscuo**; espera de 100 ms
`pcap_datalink()`	Tipo de enlace de la captura	Comprueba que es Ethernet (`DLT_EN10MB`). Con otro (loopback, SLL) los offsets cambian y se avisa
`pcap_loop()`	Llama a una función por cada paquete	Llama a `analizar_paquete()`, que hace los bloques 1 a 4
`pcap_breakloop()`	Para el bucle desde fuera	Desde el manejador de `SIGINT`: Ctrl+C termina limpio y se imprime el resumen
`pcap_geterr()`, `errbuf`	Mensajes de error	Al abrir y si `pcap_loop` falla
`pcap_close()`	Libera la captura	Al final, tras liberar la tabla de flujos

3. La cabecera de cada paquete: `caplen` frente a `len`

libpcap entrega cada paquete con un `struct pcap_pkthdr`:

- **`caplen`**: bytes que de verdad hay en memoria. Puede ser menor que el paquete si la captura se hizo con un *snaplen* corto.
- **`len`**: bytes que tenía el paquete en el cable.

Regla aplicada en todos los bloques: antes de leer cualquier cabecera (Ethernet, VLAN, IP, TCP) y antes de copiar datos al buffer de reensamblado se comprueba que `caplen` los cubre; si no, el paquete se cuenta como truncado. Es lo que impide leer fuera de la memoria con trazas cortadas.

Segunda regla: la longitud de los datos se calcula con el campo de longitud de **IP**, no con `caplen`. Ethernet rellena las tramas hasta 60 bytes, y ese relleno no son datos de TCP.

4. Relación con la práctica de AR2 (revisada en la tutoría 1)

En la práctica	En la sonda
Longitudes de trama: mínimo 60 B, máximo 1514 B	El mínimo de 60 B es el relleno de Ethernet: por eso se usa la longitud de IP y no la de pcap
Registro de funciones por EtherType	La sonda filtra a mano: IPv4 (0x0800) se analiza; ARP (0x0806) e IPv6 (0x86DD) se cuentan. Para añadir IPv6 el patrón es el mismo
Ejecución con permisos de administrador	Capturar en vivo necesita `sudo` o la capacidad `CAP_NET_RAW`; leer un fichero, no
CRC-32 y propiedad del residuo cero	No aplica: la tarjeta descarta las tramas con CRC erróneo antes de que libpcap las vea. La sonda no ve errores físicos, pero sí retransmisiones TCP, que resuelve el bloque 3

5. Lo que conviene añadir para captura en vivo

Función	Para qué
`pcap_compile()` + `pcap_setfilter()`	Filtro **BPF en el núcleo** (p. ej. `tcp port 443`): los paquetes que no interesan no llegan a la sonda. Ahora el filtro de puerto se hace en el código, después de copiar el paquete
`pcap_create()` + `pcap_set_buffer_size()` + `pcap_activate()`	Sustituyen a `pcap_open_live()` y permiten un **buffer de captura mayor**, que evita pérdidas en ráfagas de tráfico
`pcap_stats()`	Cuántos paquetes ha **descartado** el núcleo por no dar abasto. Imprescindible para saber si una medida en vivo es completa
`pcap_dump_open()` / `pcap_dump()`	Guardar en un pcap los handshakes clasificados como inseguros, como evidencia (útil para el inventario de la patente)

> ★ **PARA COMENTAR EN LA TUTORÍA**
>
> La práctica de ARP no ha aparecido, pero lo que se buscaba en ella ya está aplicado en la sonda: apertura de fichero e interfaz, comprobación del tipo de enlace, `pcap_loop` con su *callback*, parada limpia con Ctrl+C y el uso correcto de `caplen`. Para la captura en vivo faltan el filtro BPF en el núcleo y `pcap_stats()` para medir pérdidas. Pregunta: ¿se dispone del enunciado de la práctica para revisarlo?